

Resumo Programação Funcional

- **Disciplina:** Programação Funcional
 - **Professor:** Eliseu César Miguel
 - **Curso:** Ciência da Computação – UNIFAL
 - **Aluno:** Pedro Brassi Luccas
 - **Data:** Maio 2026
-

1. Introdução e Sintaxe Básica

Um programa funcional é composto por uma série de **definições de funções**, e sua execução consiste no cálculo de **expressões** baseadas nessas definições.

•

Sintaxe Básica:

- **Nomes de funções** começam com letras minúsculas (ex: `square`);
- **Nomes de tipos** começam com letras maiúsculas (ex: `Int`, `Bool`);
- O símbolo `::` significa “é do tipo...” — usado para declarar o tipo de uma função ou valor;
- **Comentários** começam com `--` (tudo à direita é ignorado pelo compilador);
- A **aplicação de função** é feita por justaposição: `f a b c` (sem parênteses nem vírgulas);
- Qualquer **operador binário** pode ser usado como função prefixada entre parênteses: `(+) 5 3 = 8`;
- Qualquer **função com 2 parâmetros** pode ser usada como operador infix entre crases: `12 `mod` 5 = 2`.

•

Exemplo Completo de Script (`example.hs`):

```
-- example.hs
module Exemplo1 where

answer :: Int
answer = 42

square :: Int -> Int
square x = x * x

allEqual :: Int -> Int -> Int -> Bool
allEqual m n p = (m == n) && (n == p)
```

```

maxi :: Int -> Int -> Int
maxi m n
  | m >= n    = m
  | otherwise = n

- answer :: Int — declara que answer é do tipo Int;
- square :: Int -> Int — função de Int para Int;
- allEqual :: Int -> Int -> Int -> Bool — recebe três Int e re-
  torna Bool;
- maxi usa equação condicional (guardas): a condição entre | e = é
  a guarda; o valor à direita do = é retornado se a guarda for verdadeira.
  otherwise funciona como else.

```

•

Como os Cálculos são Efetuados (Substituição):

A avaliação funcional funciona por **substituição de expressões** — a **ordem de avaliação não importa**, pois o resultado é sempre o mesmo (transparência referencial).

```

allEqual 2 3 3
= (2 == 3) && (3 == 3)
= False && True
= False

allEqual (square 3) answer (square 2)
= ((3*3) == 42) && (42 == (2*2))
= (9 == 42) && (42 == 4)
= False && False
= False

maxi 3 4
?? 3 >= 4 = False
?? otherwise = True
= 4

```

•

Hugs (Interpretador):

- Para testar scripts, utiliza-se o **Hugs**;
- Ao iniciar, carrega automaticamente o **Prelude.hs** (funções pré-definidas);
- Novas definições **não podem** ser criadas pela linha de comando — devem ser carregadas de arquivos;
- Comandos essenciais:

Comando	Descrição
<code>:load arquivo.hs</code>	Carrega definições de um arquivo
<code>:?</code>	Exibe lista de todos os comandos
<code>:q</code>	Sai do interpretador

– **Exemplo de sessão:**

```
Prelude> 2+3
5
Prelude> sum [1..10]
55
Prelude> reverse "hugs is cool"
"looc si sguh"
Prelude> :load example.hs
Exemplo1> maxi 3 4
4
```

- Um **módulo** é uma coleção de funções descritas em um arquivo. O nome do módulo é declarado com `module NomeModulo where`. Se não houver nome, o Hugs usa `Main`.

2. Tipos de Dados Básicos

Haskell é uma linguagem **estritamente tipada**, o que garante segurança no código — erros de tipo são detectados em tempo de compilação.

•

Números Inteiros:

- **Int**: Inteiro de precisão fixa. Valor máximo: 2147483647. Mais eficiente;
- **Integer**: Inteiros de precisão arbitrária (sem limite de tamanho);
- **Operações aritméticas**: `+`, `-`, `*`, `/` (divisão real), `div` (divisão inteira), `mod` (resto), `abs` (valor absoluto);
- **Comparação**: `>`, `<`, `>=`, `<=`, `==` (igual), `/=` (diferente).

```
-- Exemplos no Hugs:
-- Main> div 22 5  => 4
-- Main> mod 12 5  => 2
-- Main> 10 / 3    => 3.33333...
-- Main> (+) 5 3   => 8
-- Main> 12 `mod` 5 => 2
```

- **Importante**: `div` e `mod` são para divisão inteira. O operador `/` retorna ponto flutuante mesmo para inteiros.

Exemplo completo: função totalSales

```
sales :: Int -> Int
sales x
  | x == 0 = 12
  | x == 1 = 20
  | x == 2 = 18
  | x == 3 = 25
  | otherwise = 0

totalSales :: Int -> Int
totalSales n
  | n == 0    = sales 0
  | otherwise = totalSales (n - 1) + sales n
```

Avaliação de totalSales 2:

```
totalSales 2
= totalSales 1 + sales 2
= (totalSales 0 + sales 1) + sales 2
= (sales 0 + sales 1) + sales 2
= (12 + 20) + 18 = 50
```

Exemplo: função maxSales (cuidado com ineficiência!)

-- Versão ineficiente: maxSales (n-1) calculado 2 vezes

```
maxSales :: Int -> Int
maxSales n
  | n == 0                = sales 0
  | maxSales (n-1) >= sales n = maxSales (n-1)
  | otherwise              = sales n
```

-- Versão eficiente e clara (usa a função maxi):

```
maxSales :: Int -> Int
maxSales n
  | n == 0    = sales 0
  | otherwise = maxi (maxSales (n-1)) (sales n)
```

Aliases de Tipo com type:

```
type Dia    = Int
type Venda = Int
```

```
f :: Dia -> Venda
f 1 = 9
f 2 = 7
f x = -1
```

O `type` cria sinônimos de tipo — melhora legibilidade sem criar um tipo novo.

•

Booleanos (Bool):

- Valores: `True`, `False`;
- **Operações:** `&&` (E lógico), `||` (OU lógico), `not` (negação);
- Operadores de comparação (`>`, `<`, `==`, etc.) já **retornam** `Bool` — nunca é necessário checar se o resultado é `True` ou `False` explicitamente;
- **Tabela verdade:**

<code>&&</code>	<code>True</code>	<code>False</code>	<code>\\ </code>	<code>True</code>	<code>False</code>
True	True	False	True	True	True
False	False	False	False	True	False

- **Boa Prática:** Evite `if cond then True else False`. Use diretamente a expressão booleana:

```
-- Ruim (redundante):
aMaiorQueB :: Int -> Int -> Bool
aMaiorQueB a b
  | a > b      = True
  | otherwise = False
```

```
-- Bom (direto):
aMaiorQueB :: Int -> Int -> Bool
aMaiorQueB a b = a > b
```

- **XOR com casamento de padrões:**

```
xor :: Bool -> Bool -> Bool
xor True  False = True
xor False True  = True
xor _     _     = False
-- xor True True  = False
-- xor False True = True
```

•

Caracteres e Strings:

- **Char**: Um único caractere Unicode entre aspas simples (ex: 'a', 'Z', '7');
- **String**: Sinônimo de [Char] (lista de caracteres). "abc" é equivalente a ['a','b','c'];
- **Módulo Data.Char** — incluir com `import Data.Char` no topo do arquivo:

Função	Descrição	Exemplo
<code>isUpper c</code>	True se c for maiúsculo	<code>isUpper 'A' = True</code>
<code>isLower c</code>	True se c for minúsculo	<code>isLower 'b' = True</code>
<code>isDigit c</code>	True se c for dígito	<code>isDigit '3' = True</code>
<code>isAlphaNum c</code>	True se c for alfanumérico	<code>isAlphaNum 'a' = True</code>
<code>ord c</code>	Código ASCII/Unicode de c	<code>ord 'a' = 97</code>
<code>chr n</code>	Caractere correspondente ao código n	<code>chr 97 = 'a'</code>

3. Estruturas de Dados: Listas e Tuplas

•

Listas ([t]):

Coleção **ordenada** de elementos do **mesmo tipo**. Ordem e repetição importam: `[1,2] /= [2,1]` e `[1,1,2] /= [1,2]`.

```
[1,2,3]      :: [Int]
[True,False] :: [Bool]
['a','b','c'] :: [Char]    -- equivale a "abc"
[]           :: [t]        -- lista vazia (polimórfica)
[[1,2],[3]]  :: [[Int]]    -- lista de listas
```

—

Construção:

* **Operador Cons (:)** — adiciona um elemento ao **início** da lista.

Tipo: `t -> [t] -> [t]`:

```
1 : [2,3]    = [1,2,3]
'a' : "bc"    = "abc"
1 : 2 : 3 : [] = [1,2,3]    -- forma canônica
-- ERRO: [] : 1 : 2         -- [] não é um elemento Int
```

```

* Concatenação (++) — une duas listas do mesmo tipo. Tipo:
[t] -> [t] -> [t]:

[1,2] ++ [3,4] = [1,2,3,4]
"ola" ++ "!"   = "ola!"
-- Diferença: 1:[2,3] e [1]++[2,3] dão o mesmo resultado,
-- mas são operações distintas (: é mais eficiente)

* Intervalos (Ranges):

[1..5]      = [1,2,3,4,5]
[1,3..9]    = [1,3,5,7,9]  -- com passo
[10,8..0]   = [10,8,6,4,2,0] -- decrescente

```

—

Acesso e Casamento de Padrão:

- * **head** — primeiro elemento; **tail** — resto da lista (sem o primeiro);
- * O padrão **(x:xs)** desestrutura uma lista: **x** é a cabeça, **xs** é a cauda;
- * O padrão **[]** casa com a lista vazia.

```

head' :: [a] -> a
head' (a:_) = a
-- head' [1,2,3,4] = 1

tail' :: [a] -> [a]
tail' (_:x) = x
-- tail' [1,2,3,4] = [2,3,4]

```

—

Funções Recursivas sobre Listas:

O padrão geral de recursão em listas é: **caso base** para **[]** e **caso recursivo** para **(x:xs)**.

```

-- Soma dos elementos
sumList :: [Int] -> Int
sumList []      = 0
sumList (x:xs) = x + sumList xs
-- sumList [1,3,2] = 1 + 3 + 2 + 0 = 6

-- Dobrar cada elemento
double :: [Int] -> [Int]
double []      = []
double (x:xs) = (2*x) : double xs
-- double [5,3] = [10,6]

```

```

-- Verificar pertencimento
membro :: Int -> [Int] -> Bool
membro _ [] = False
membro x (y:ys)
  | x == y = True
  | otherwise = membro x ys
-- membro 3 [5,3,2] = True
-- membro 4 [5,3,2] = False

```

•

Tuplas ((t1, t2, ..., tn)):

Coleção de **tamanho fixo** que pode conter elementos de **tipos diferentes**.

```

(1, 'a') :: (Int, Char)
("Joao", 25) :: (String, Int)
(True, "a", 3.0) :: (Bool, String, Float)

```

– **Funções de Par** (apenas para tuplas de 2 elementos):

```

fst (1, 'a') = 1      -- retorna o primeiro elemento
snd (1, 'a') = 'a'    -- retorna o segundo elemento

```

– **Casamento de Padrão em Tuplas:**

```

somaPar :: (Int, Int) -> Int
somaPar (x, y) = x + y
-- somaPar (3, 4) = 7

-- Extrair componentes com _ (wildcard):
dados = ("Alice", 30)
nome (n, _) = n
idade (_, i) = i
-- nome dados = "Alice"
-- idade dados = 30

```

4. Lógica de Programação: Recursão e Casamento de Padrões

Como Haskell **não possui loops** (for/while), a **recursão** é a principal ferramenta de iteração.

•

Casamento de Padrões (Pattern Matching):

Permite definir funções para diferentes “formatos” de entrada. Os padrões são testados **de cima para baixo** — o primeiro que casar é usado.

```
-- Em listas:
length' :: [a] -> Int
length' [] = 0
length' (_:xs) = 1 + length' xs

-- Em tuplas:
somaPar :: (Int, Int) -> Int
somaPar (x, y) = x + y

-- Em valores diretos (literais):
sales :: Int -> Int
sales 0 = 12
sales 1 = 20
sales 2 = 18
sales _ = 0    -- _ é wildcard (casa com qualquer coisa)
```

•

Equações Condicionais (Guardas):

Utilizadas quando a escolha depende de uma **condição sobre os argumentos**, não apenas de sua estrutura.

```
bmi :: Float -> String
bmi b
  | b < 18.5 = "Abaixo do peso"
  | b < 25.0 = "Normal"
  | b < 30.0 = "Sobrepeso"
  | otherwise = "Obeso"
```

- O símbolo `|` introduz cada guarda;
- `otherwise` é equivalente a `True` — funciona como o `else` final;
- As guardas são avaliadas **de cima para baixo**.

•

Recursão: Estrutura Geral

Todo algoritmo recursivo precisa de:

1. **Caso base** — condição de parada (ex: lista vazia, `n == 0`);
2. **Caso recursivo** — reduz o problema e chama a si mesmo.

```
-- Fatorial
fatorial :: Int -> Int
```

```

fatorial 0 = 1
fatorial n = n * fatorial (n - 1)
-- fatorial 4 = 4 * 3 * 2 * 1 * 1 = 24

-- Fibonacci
fib :: Int -> Int
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)

```

5. Funções de Alta Ordem (Higher-Order Functions)

Funções de alta ordem são aquelas que **recebem funções como argumento** e/ou **retornam funções como resultado**. Permitem criar abstrações poderosas e código reutilizável.

-- Exemplo conceitual de assinatura:
operarSobreLista :: (a -> b) -> [a] -> [b]

•

map: Aplica uma função a cada elemento de uma lista.

Assinatura: map :: (a -> b) -> [a] -> [b]

```

map (+1)    [1,2,3]      = [2,3,4]
map (*2)    [1,2,3]      = [2,4,6]
map even    [1,2,3]      = [False,True,False]
map reverse ["ola","amor"] = ["alo","roma"]

```

– Equivalência com recursão explícita:

```

-- Sem map:
double :: [Int] -> [Int]
double []      = []
double (x:xs) = (2*x) : double xs

```

```

-- Com map (mais conciso):
double xs = map (*2) xs

```

•

filter: Filtra elementos que satisfazem um predicado.

Assinatura: filter :: (a -> Bool) -> [a] -> [a]

```

filter even [1,2,3,4,5] = [2,4]
filter (>3) [1,5,2,8]  = [5,8]

```

```
filter (<3)    [1,5,2,8]    = [1,2]
filter isUpper "HaSKell"    = "HSK"
```

•

fold (Dobradura): Reduz uma lista a um único valor.

A ideia do fold é **combinar todos os elementos** de uma lista usando uma função binária e um valor inicial (acumulador).

Definição genérica:

```
fold :: (t -> u -> u) -> u -> [t] -> u
fold f s []      = s
fold f s (a:x)   = f a (fold f s x)
```

—

foldr (fold right) — processa da direita para a esquerda:

Assinatura: foldr :: (a -> b -> b) -> b -> [a] -> b

```
foldr (+) 0 [1,2,3]
= 1 + (foldr (+) 0 [2,3])
= 1 + (2 + (foldr (+) 0 [3]))
= 1 + (2 + (3 + (foldr (+) 0 [])))
= 1 + (2 + (3 + 0))
= 6
```

—

foldl (fold left) — processa da esquerda para a direita:

Assinatura: foldl :: (b -> a -> b) -> b -> [a] -> b

```
foldl (+) 0 [1,2,3]
= foldl (+) (0+1) [2,3]
= foldl (+) (1+2) [3]
= foldl (+) (3+3) []
= 6
```

—

Funções clássicas implementadas com fold:

```
sum'      xs = foldr (+) 0 xs    -- soma
product'  xs = foldr (*) 1 xs    -- produto
and'      xs = foldr (&&) True xs
or'       xs = foldr (||) False xs
```

-- concat com foldr:

```
concat' :: [[a]] -> [a]
```

```

concat' = foldr (++) []
-- concat' [[1,2],[3],[],[4,5]] = [1,2,3,4,5]

-- reverse com foldl:
reverse' :: [a] -> [a]
reverse' = foldl (flip (::)) []
-- reverse' [1,2,3] = [3,2,1]

```

Diferença prática entre foldr e foldl: Para operações associativas como + e *, o resultado é o mesmo. A diferença aparece em operações não-comutativas:

```

foldr (-) 0 [1,2,3] = 1-(2-(3-0)) = 1-2+3 = 2
foldl (-) 0 [1,2,3] = ((0-1)-2)-3 = -6

```

6. Funções Úteis do Prelude

O **Prelude** é carregado automaticamente pelo Hugs/GHCi e contém diversas funções essenciais:

Função	Tipo	Descrição	Exemplo
head	[a] -> a	Primeiro elemento	head [1,2,3] = 1
tail	[a] -> [a]	Resto da lista	tail [1,2,3] = [2,3]
length	[a] -> Int	Tamanho da lista	length [1,2,3] = 3
reverse	[a] -> [a]	Inverte a lista	reverse [1,2,3] = [3,2,1]
sum	[Int] -> Int	Soma os elementos	sum [1,2,3] = 6
product	[Int] -> Int	Produto dos elementos	product [1,2,3] = 6
null	[a] -> Bool	Testa se a lista é vazia	null [] = True
elem	a -> [a] -> Bool	Testa pertencimento	elem 3 [1,2,3] = True
zip	[a]->[b]->[(a,b)]	Emparelha duas listas	zip [1,2] ['a','b'] = [(1,'a'),(2,'b')]
fst	(a,b) -> a	Primeiro de um par	fst (1,'a') = 1

Função	Tipo	Descrição	Exemplo
snd	(a,b) -> b	Segundo de um par	snd (1,'a') = 'a'
even	Int -> Bool	Testa se é par	even 4 = True
odd	Int -> Bool	Testa se é ímpar	odd 3 = True

7. Exercícios das Aulas (com soluções)

Aula 02 — Números Inteiros

```

type Dia    = Int
type Venda = Int

f :: Dia -> Venda
f 1 = 9; f 2 = 7; f 3 = 15; f 4 = 14
f 5 = 8; f 6 = 0; f 7 = 3; f _ = -1

-- 1. Dia da maior venda
diaMaiorVenda :: Int -> Int
diaMaiorVenda 1 = 1
diaMaiorVenda n
  | f n > f (diaMaiorVenda (n-1)) = n
  | otherwise                    = diaMaiorVenda (n-1)

-- 2. Total de vendas no período
totalVendas :: Int -> Int
totalVendas 0 = 0
totalVendas n = totalVendas (n-1) + f n

-- 3. Média de vendas
mediaVendas :: Int -> Int
mediaVendas n = totalVendas n `div` n

```

Aula 03 — Listas e Tuplas

```

vendas :: Int -> Int
vendas 0=0; vendas 1=41; vendas 2=72; vendas 3=48
vendas 4=0; vendas 5=91; vendas 6=55; vendas 7=30

-- 1. Lista de vendas
listaVendas :: Int -> [Int]
listaVendas 0 = [vendas 0]

```

```

listaVendas n = listaVendas (n-1) ++ [vendas n]

-- 4. Lista de tuplas (dia, venda)
listaTuplas :: Int -> [(Int,Int)]
listaTuplas 0 = [(0, vendas 0)]
listaTuplas n = listaTuplas (n-1) ++ [(n, vendas n)]

-- 5. Dia da maior venda
diaMaior :: Int -> Int
diaMaior 0 = 0
diaMaior n
  | vendas n > vendas (diaMaior (n-1)) = n
  | otherwise                          = diaMaior (n-1)

```

Aula 04 — Char e Bool

```

import Data.Char

-- 1. Separar [(Int,Char)] em ([Int],[Char])
separar :: [(Int,Char)] -> ([Int],[Char])
separar [] = ([], [])
separar ((n,c):xs) = (n:ns, c:cs)
  where (ns, cs) = separar xs

-- 2. Marcar alfanuméricos
marcaAlfa :: [Char] -> [(Bool,Char)]
marcaAlfa [] = []
marcaAlfa (c:cs) = (isAlphaNum c, c) : marcaAlfa cs

-- 5. Gerar tabela ASCII (32 a 126)
tabelaASCII :: [(Int,Char)]
tabelaASCII = zip [32..126] (map chr [32..126])

```

Aula 05 — Funções de Alta Ordem

```

-- 1. Quadrado de cada elemento
quadrados :: [Int] -> [Int]
quadrados = map (^2)
-- quadrados [1,2,3,4] = [1,4,9,16]

-- 2. Filtrar múltiplos de 5
multiplosDe5 :: [Int] -> [Int]
multiplosDe5 = filter (\x -> x `mod` 5 == 0)
-- multiplosDe5 [1..20] = [5,10,15,20]

-- 3. Quadrado dos pares

```

```

quadradoDosPares :: [Int] -> [Int]
quadradoDosPares xs = map (^2) (filter even xs)
-- quadradoDosPares [1..6] = [4,16,36]

-- 4. concat com foldr
concat' :: [[a]] -> [a]
concat' = foldr (++) []
-- concat' [[1,2],[3],[],[4,5]] = [1,2,3,4,5]

-- 5. reverse com foldl
reverse' :: [a] -> [a]
reverse' = foldl (flip (:)) []
-- reverse' [1,2,3] = [3,2,1]

```

8. Conceitos Avançados: Funções Anônimas (Lambda)

Haskell permite definir funções sem nome (expressões lambda) usando `\`:

```

-- \x -> x + 1   é equivalente a   (+1)
map (\x -> x * x) [1,2,3]  = [1,4,9]
filter (\x -> x > 2 && even x) [1..10] = [4,6,8,10]

-- Lambdas com múltiplos argumentos:
(\x y -> x + y) 3 4 = 7

```

9. Conceitos Avançados: Aplicação Parcial e Currying

Em Haskell, **todas as funções são curried** — uma função de múltiplos argumentos é, na verdade, uma sequência de funções de um argumento. Isso permite **aplicação parcial**:

```

-- add é uma função que recebe Int e retorna (Int -> Int):
add :: Int -> Int -> Int
add x y = x + y

add3 :: Int -> Int
add3 = add 3    -- aplicação parcial: fixamos o primeiro argumento

add3 5 = 8
add3 10 = 13

-- Exemplos com map:
map (add 10) [1,2,3] = [11,12,13]
map (div 100) [5,10,20] = [20,10,5]

```

10. Tabela-Resumo: Quando Usar Cada Estrutura

Situação	Estrutura	Motivo
Elementos do mesmo tipo, tamanho variável	Lista <code>[t]</code>	Recursão natural, funções como <code>map/filter/fold</code>
Elementos de tipos diferentes, tamanho fixo	Tupla <code>(t1,t2)</code>	Agrupa dados heterogêneos sem criar novo tipo
Valores inteiros eficientes	<code>Int</code>	Precisão fixa, mais rápido
Inteiros muito grandes	<code>Integer</code>	Sem limite de tamanho
Caractere único	<code>Char</code>	Entre aspas simples
Texto	<code>String = [Char]</code>	Lista de caracteres
Valor lógico	<code>Bool</code>	<code>True</code> ou <code>False</code>